

# Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

## Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify

## { T }
```

## Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, imitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y was converted from Y = 1/1,000 years
myFunc <- function(x) {
  case_when(
    0 <= x & x <= (-9+ 16)/(13.6) ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x & x <= (-7.500 + 16)/(13.6) ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x & x <= (-4.3+ 16)/(13.6) ~ 3,
    (-4.3 + 16)/(13.6) < x & x <= (-3.000+ 16)/(13.6) ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x & x <= (-2.400+ 16)/(13.6) ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE ~ 1
  )
}
curve(myFunc(x), col = 2)

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
  T[i]<-rpois(1, Y[i]* 25)
  if(T[i] < 1) {T[i]<- 1}
  else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
  SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL)
```

```

plot(Y, T)
plot(Y, SL)

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5

## [1] 0.1552983 0.4762450

plot(density(S))

plot(dS, S)
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S)
plot(Y, S)
plot(T, dS) # no correlation
plot(SL, S)

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i] - dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K) # try to make less than 0.5

## [1] 0.05886012 0.43876435

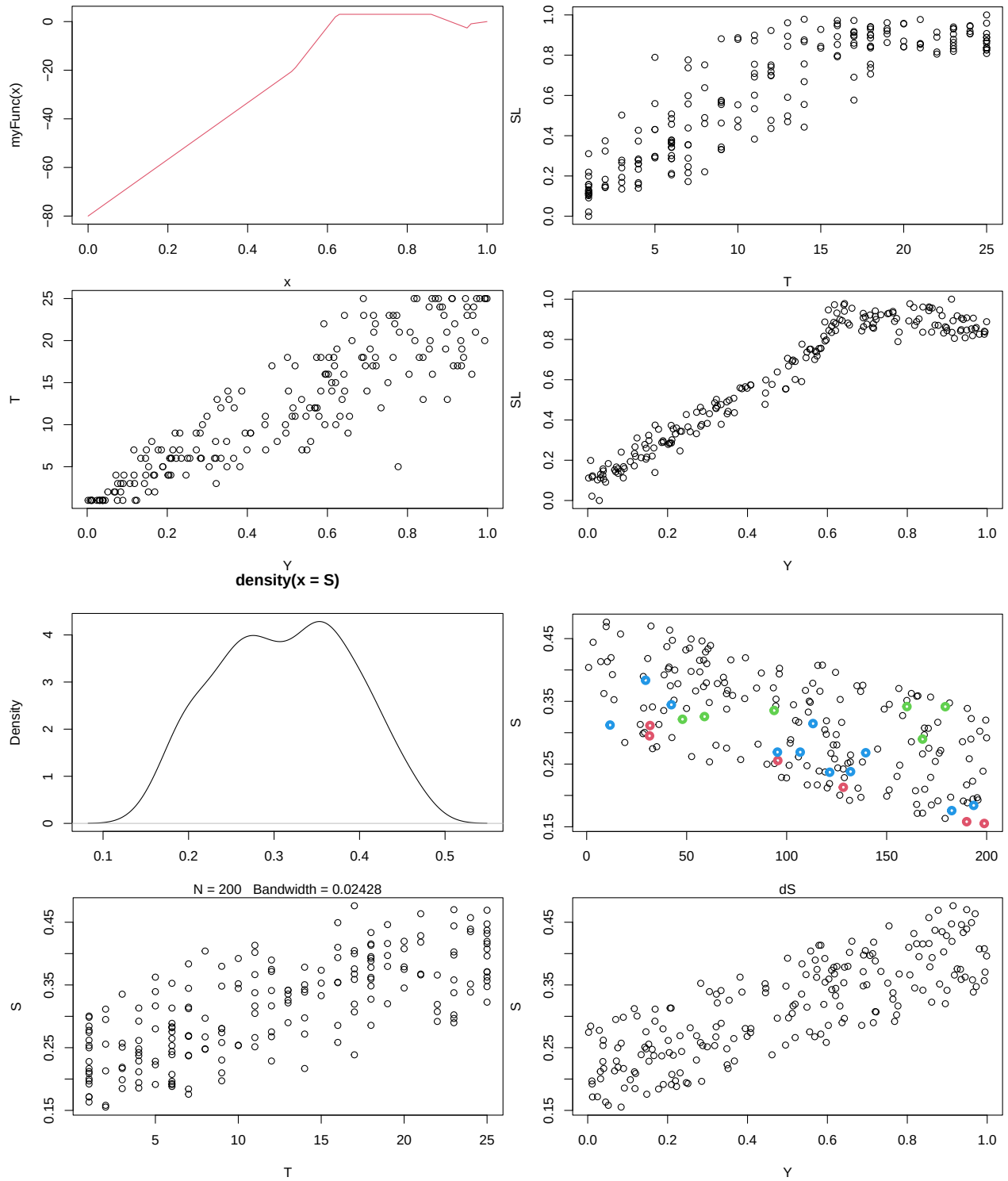
plot(density(K))
plot(dK, K)
plot(SL, K)
plot(Y, K)
plot(S, K)
plot(dS, K) # no correlation

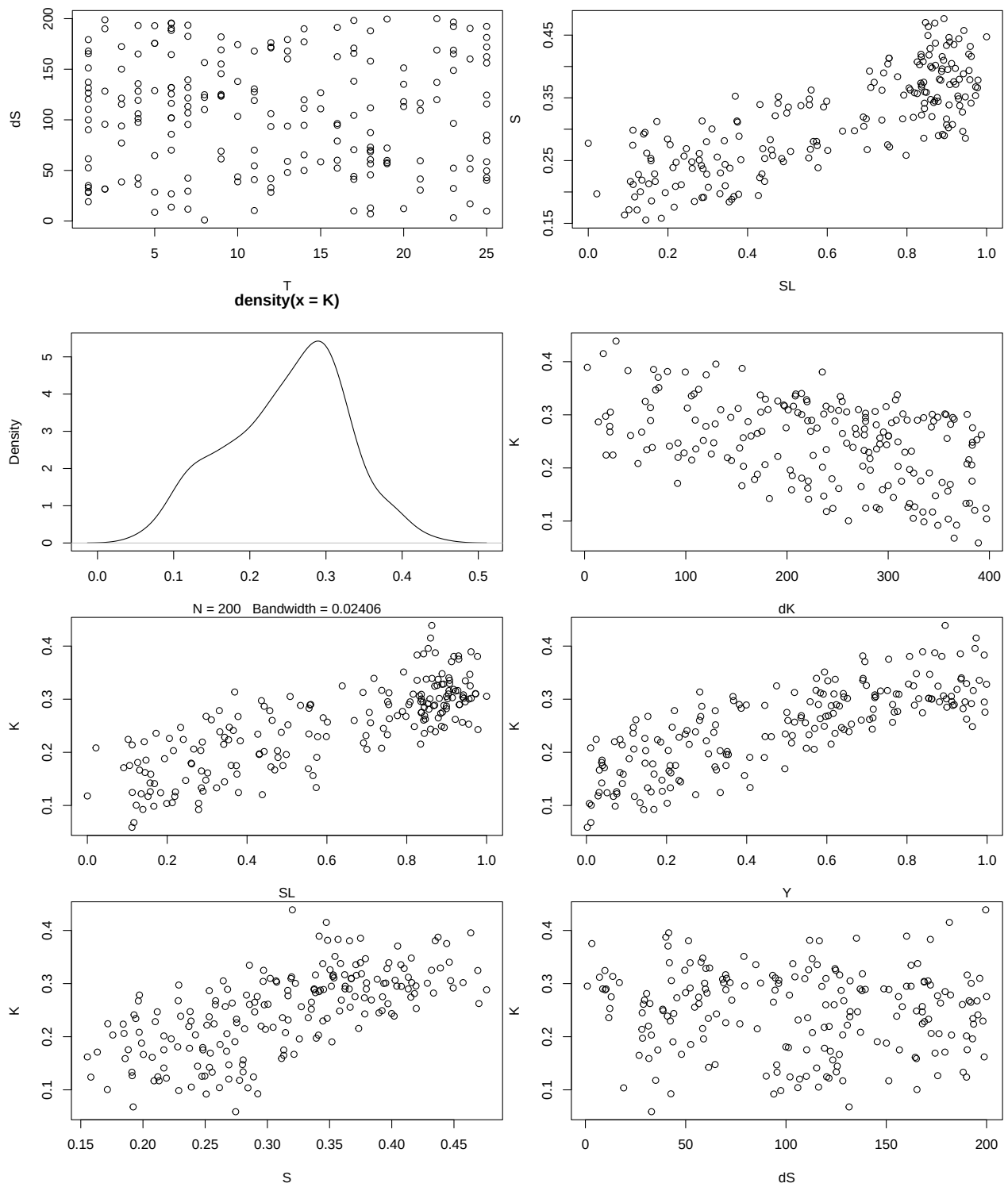
# Standardize data without SL, S, K (those are ratio)
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))

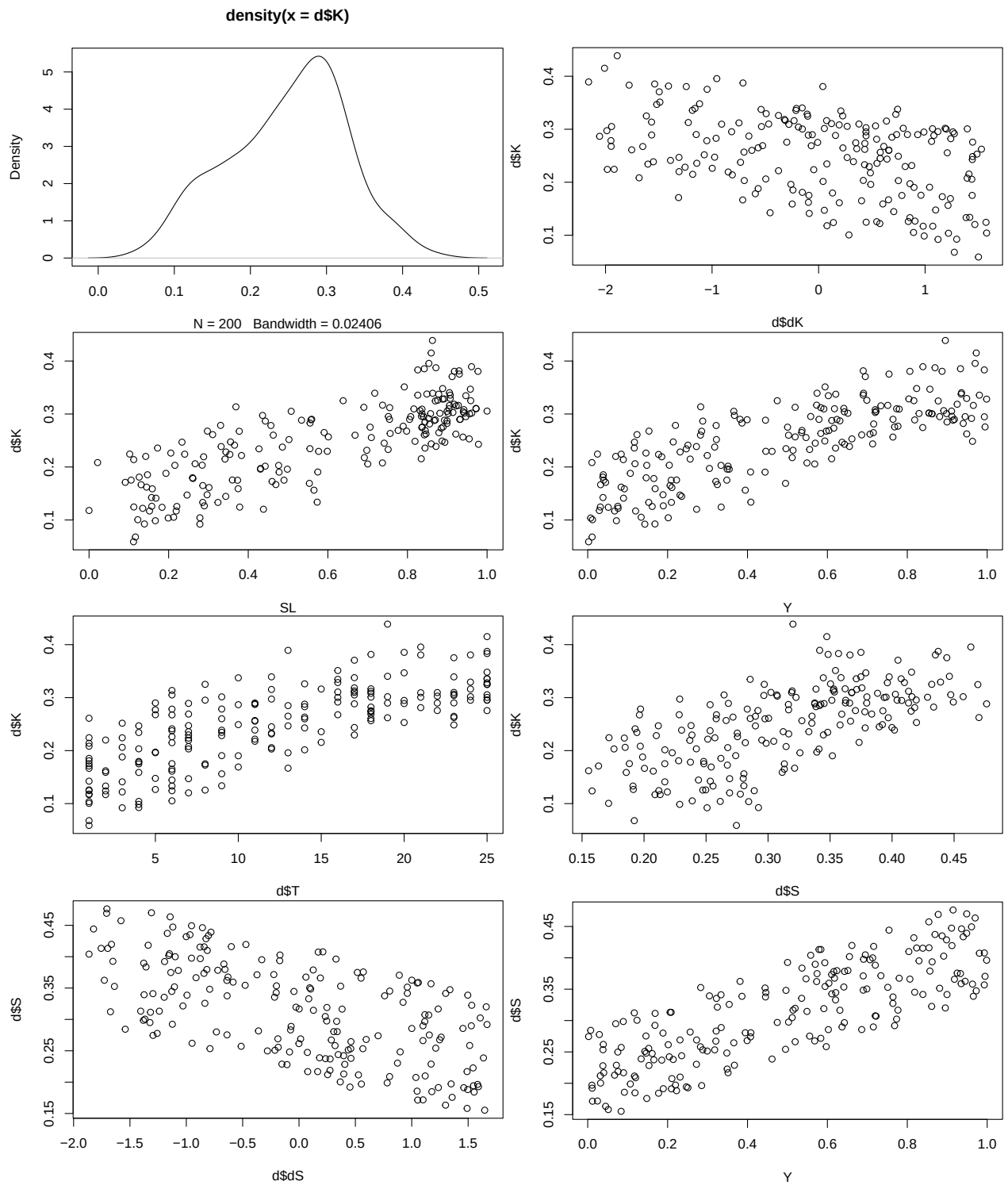
# check the data
plot(density(d$K))
plot(d$dK, d$K)
plot(SL, d$K)
plot(Y, d$K)
plot(d$T, d$K)
plot(d$S, d$K)
plot(d$dS, d$S)
plot(Y, d$S)

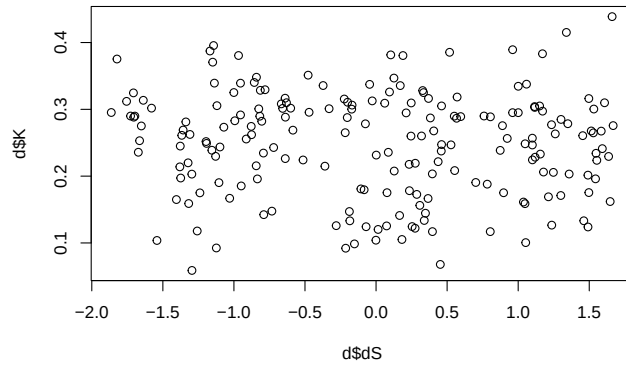
```

```
plot(d$dS, d$K) # no correlation
```





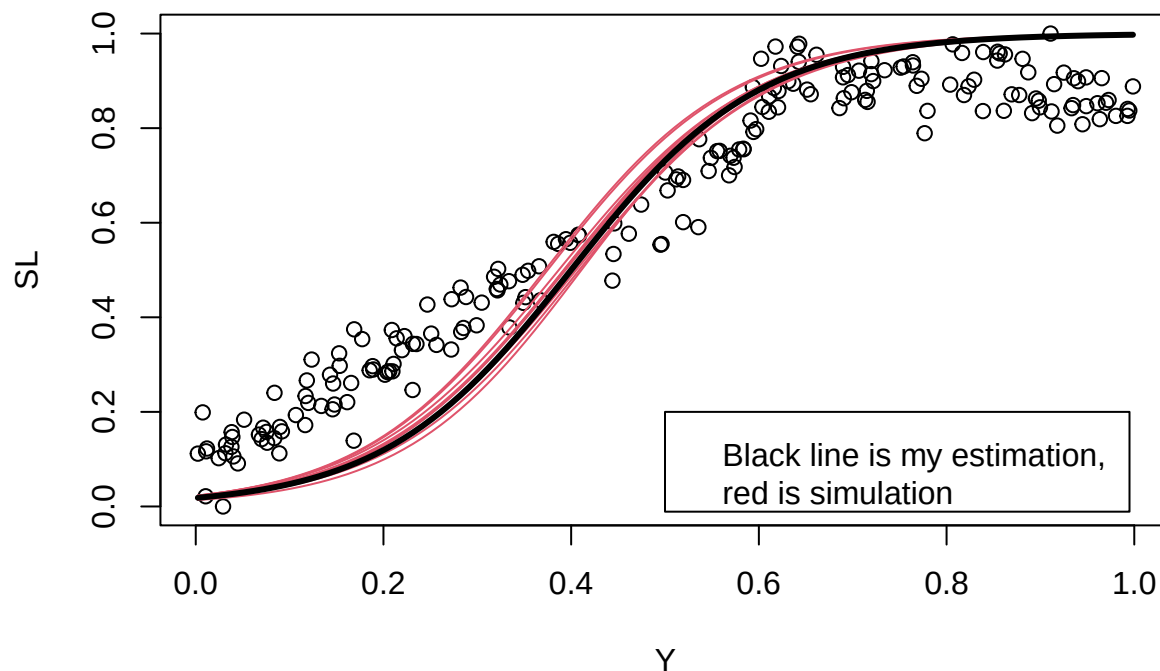




## Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and SL are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL)
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 10, 0.2)*x + rnorm(1, -4, 0.2)), add =TRUE, col = 2)
curve(inv_logit(10*x - 4), add =TRUE, lwd = 3)
legend(0.5, 0.2,legend="Black line is my estimation, \nred is simulation")
```



## Model 1

- Also see overview.pdf to know statistical model and prior
- There are a few messages to warning my priors.
- This would be due to the prior of Y, which is very weakly initialized. Since it only shows up during first iteration of warmup, we can safely ignore it.
- If you really fix it, you can use normal distribution with large sigma and constraints between -16 and -2.4

```

# Model 1
m1<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # We do not know anything about this but think similar for all group.
    # So use partial pooling to regularize the estimate.
    # mean should start at 0 and has small variation.

    # In this case (m1), we do not include correlation.
    # This is because multilevel models of S and SL includes the information of Y
    # But we have to regularize.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_S_bar, bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    # don't need regularization as dK is not affected by T
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.5),
    U ~ exponential(1),

# Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_dS * dS + bS_Y[T] * Y[i],

    # I am hoping/believing b_dS is negative and b_Y_S is positive
    # partial pooling for the same reason for a2 and b_Y_S
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.1),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.05),
    zbS_Y[T] ~ normal(0,0.05),

    # don't need regularization
    bS_dS ~ normal(0,0.1),
    sigma_S ~ exponential(1),

# Model SL by Y
    # The log function and prior was refered to the previous research of Sea Level Change over
    # This function is abstract so I will have to actually stan code to provide accurate estima
    # ulam code in r does not allow us to hard code the conditional function.
    # vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),
    vector[200]: SL ~ normal(mu_SL, sigma_SL),

```

```

vector[200]: logit(mu_SL) <- aSL + Y[i] * bSL,
  # Should be very low variation since this part is hard coded.
  # This time the estimation is not great so this is fine
aSL ~ normal(-4, 0.2),
bSL ~ normal(10, 0.2),

sigma_SL ~ exponential(1),

  # Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

  # Model T by Y
vector[200]:Y ~ uniform( 0, 1 ) # This this is something to do with the +/- change

), dat = d, constraints = list(
  SL = "lower=0,upper=1" # SL is between 0 and 1
  #, T = "lower=1,upper=25" #this is unnecessary since T is provided in the dat
  # 4000 iteration and 2000 warmup to efficiently sample
), iter = 4000, warmup = 2000, chains=4, cores=4, log_lik=TRUE
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:1994:3: note: by 'void ulam_cmd
## 1994 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:1994:3: note: by 'void ulam_cmd
## 1994 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:2031:3: note: by
## 'T_ ulam_cmdstanr_fcb02887c505f24963feee71d86c1eac_model_namespace::ulam_cmdstanr_fcb02887c505f24963
## 2031 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:2031:3: note: by 'T_ ulam_cmdsta
## 2031 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:2031:3: note: by 'T_ ulam_cmdsta
## T_a, -1, 1>&, std::ostream*) const'

```



```

## 2031 | log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp::
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a03577d14.hpp:2031:3: note: by 'T_ ulam_cmdsta
## 2031 | log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 1
## Chain 2 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 2
## Chain 3 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 3
## Chain 4 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 4
## Chain 3 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 200 / 4000 [ 5%] (Warmup)

```

```

## Chain 4 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)

```

[illegible]

```

## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 342.1 seconds.
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 354.4 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 354.6 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 355.3 seconds.
##

```

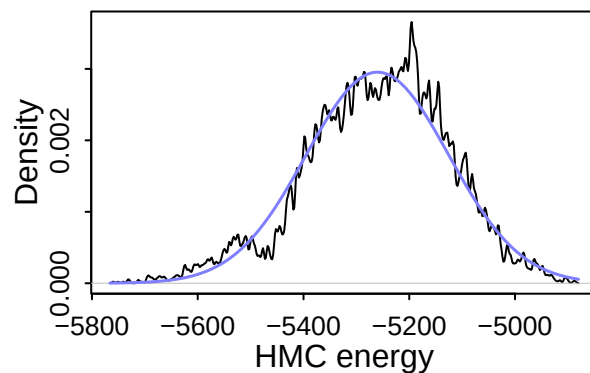
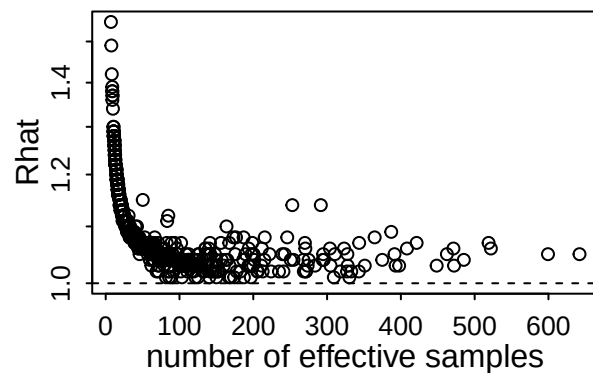
```
## All 4 chains finished successfully.
## Mean chain execution time: 351.6 seconds.
## Total execution time: 355.6 seconds.

## Warning: 167 of 8000 (2.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 7833 of 8000 (98.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

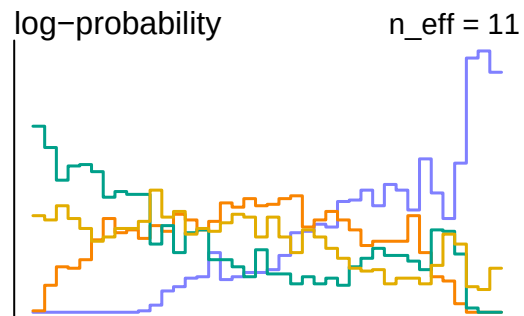
```
dashboard(m1)
```



# 167

Divergent transitions

Check yourself before  
you wreck yourself



```
precis(m1, depth=1)
```

```
## 550 vector or matrix parameters hidden. Use depth=2 to show them.

##           mean      sd      5.5%      94.5%      rhat
## bK_Y_bar  0.497765520 0.115622890 0.3037611746 0.680633580 1.076492
## bK_SL_bar 0.440691704 0.098456107 0.2738155991 0.601073358 1.123121
## bK_S_bar  0.550070080 0.203365562 0.2060963991 0.858751661 1.202935
## aK_bar    -1.823137268 0.055659542 -1.9082871975 -1.732654625 1.271306
## sigma_bK_Y 0.168896842 0.142968182 0.0109677640 0.432668271 1.032157
## sigma_bK_SL 0.135982462 0.126418424 0.0091128986 0.386947172 1.066738
## sigma_bK_S 0.168286306 0.152764263 0.0144195667 0.450201615 1.028694
## sigma_aK   0.178729004 0.134464190 0.0181607315 0.433333100 1.050274
## bK_dK      -0.219300350 0.009962348 -0.2354178723 -0.203369286 1.063965
## U          0.023625969 0.001346346 0.0216367473 0.025787475 1.036911
## bS_Y_bar   0.913612540 0.029280680 0.8722340482 0.962869895 1.206859
## aS_bar     -1.263303451 0.016075311 -1.2875874650 -1.236011624 1.090509
## sigmabS_Y  0.161769010 0.145298259 0.0115073701 0.430196981 1.014658
## sigma_aS   0.204083386 0.159686781 0.0145460749 0.512509095 1.039700
```

```
## bS_dS      -0.220014374  0.007113598 -0.2310225972 -0.208164128  1.049832
## sigma_S    0.011079285  0.001934471  0.0078802280  0.013855486  1.252453
## aSL        -4.011553966  0.168667338 -4.2316361140 -3.705274380  1.115771
## bSL        9.963429163  0.181698845  9.6977617715 10.255331750  1.059615
## sigma_SL   0.001097971  0.000719552  0.0002416731  0.002415316  1.373165
##           ess_bulk
## bK_Y_bar    40.198962
## bK_SL_bar   24.739430
## bK_S_bar    14.901173
## aK_bar      11.550555
## sigma_bK_Y  103.602414
## sigma_bK_SL 48.917463
## sigma_bK_S  193.294664
## sigma_aK    148.118986
## bK_dK       63.582040
## U           83.599984
## bS_Y_bar    14.996379
## aS_bar      37.339395
## sigmabS_Y   169.427000
## sigma_aS    99.291149
## bS_dS       90.551164
## sigma_S     12.298661
## aSL         31.039000
## bSL         76.817083
## sigma_SL    9.377356
```

- This has so much divergent and less effective sampling.
- This would be due to uniform distribution (little informative)

## Posterior Check

```
post <- extract.samples(m1)
```

## Hidden parameters

- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y)
points(1:200, post$Y[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

plot(Y, d$K)
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

plot(SL, d$K)
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

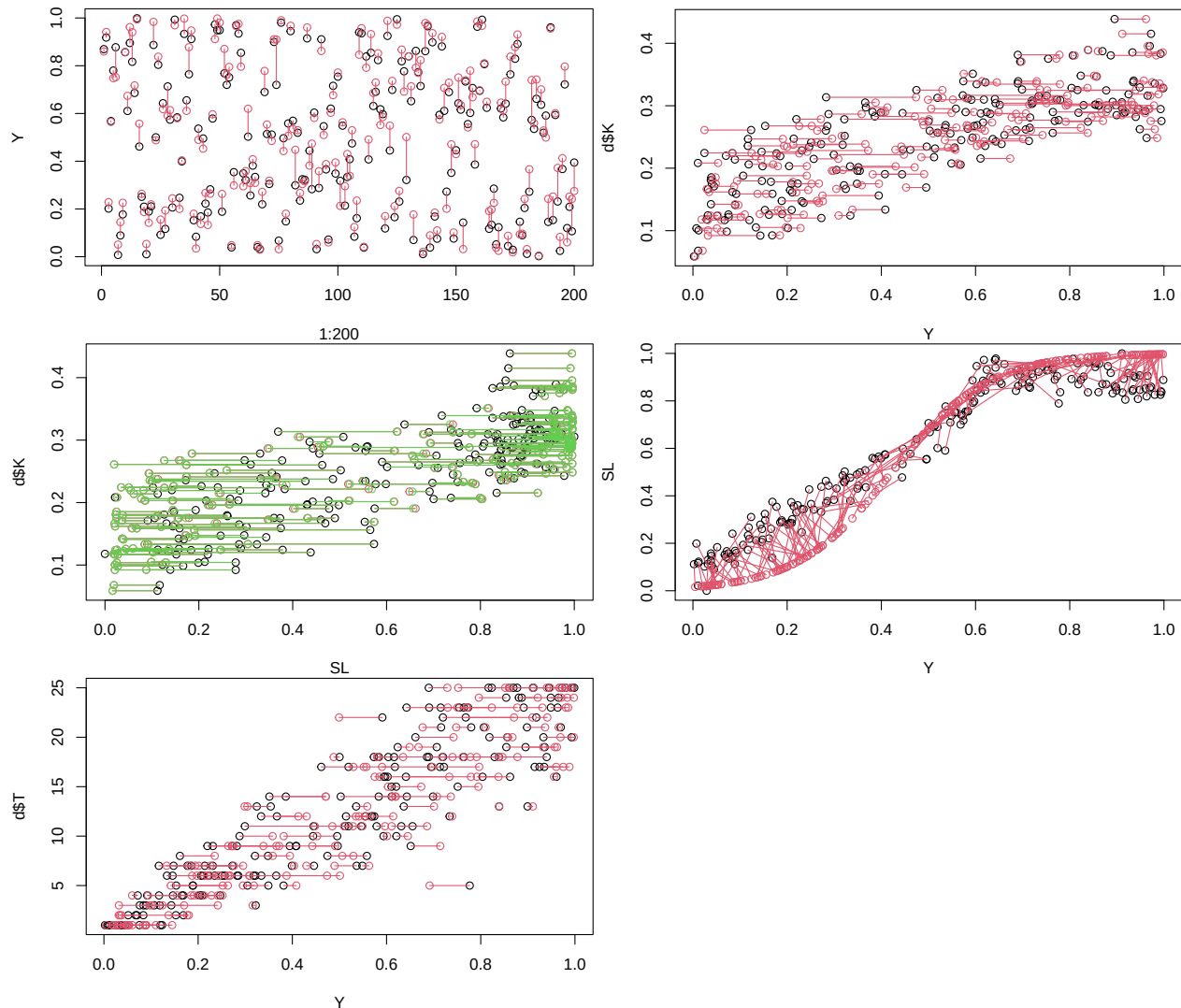
plot(Y, SL)
```

```

points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)

plot(Y, d$T)
points(post$Y[1,], d$T ,col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i],d$T[i]), col = 2)

```



=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthesized) data.

## Posterior Simulation

### Model K

```

plot(d$S, d$K)
Sseq<-seq(0, 0.5, len = 20)
KbyS<- apply(Sseq,
  function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + t(t(post$bK
means<- apply( KbyS , 2 , mean )

```

```

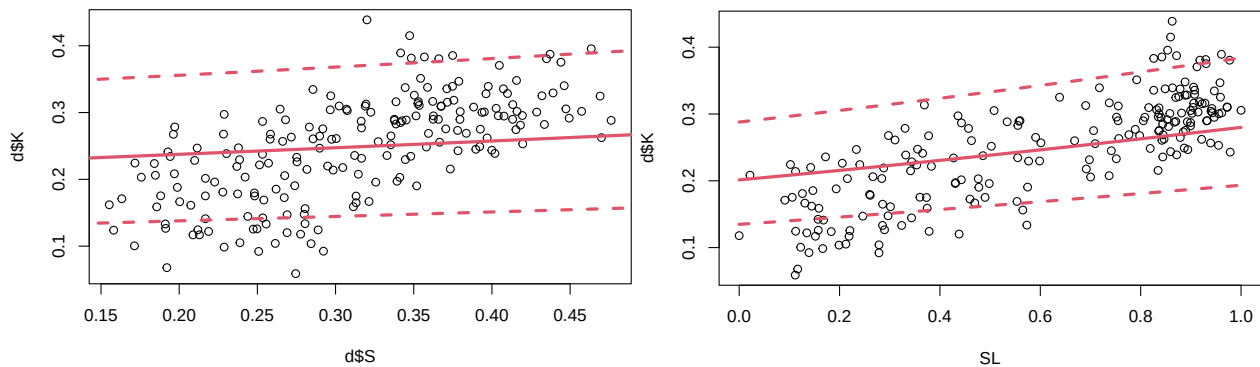
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K)
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq, # Tricky but t and t to calculate estimates by group, though dK does not have gro
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + t(t(post$bK_Y) * colMeans
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

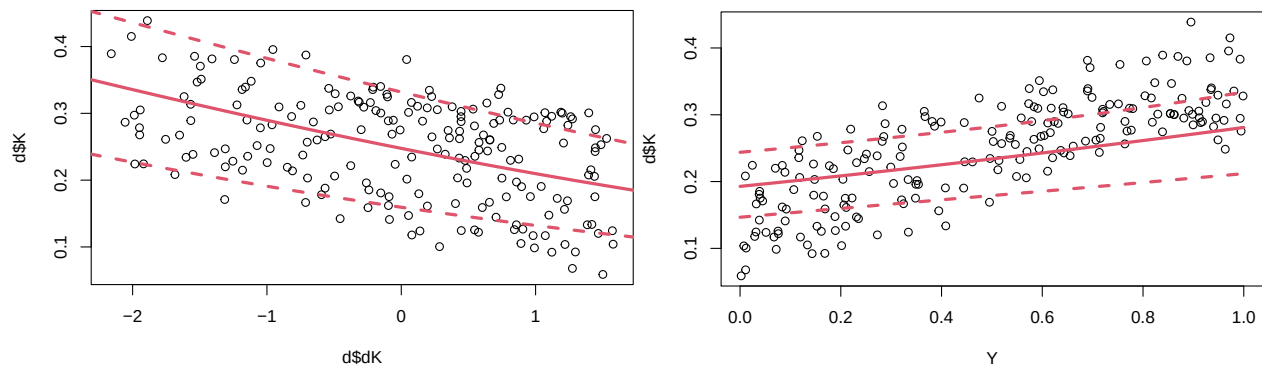
plot(d$dK, d$K)
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + t(t
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(Y, d$K)
Yseq<-seq(0, 1, len = 20)
KbyY<- sapply(Yseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + pos
PIs<- apply( KbyY , 2 , PI )
means<- apply( KbyY , 2 , mean )
lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```







### Model S by dS and Y

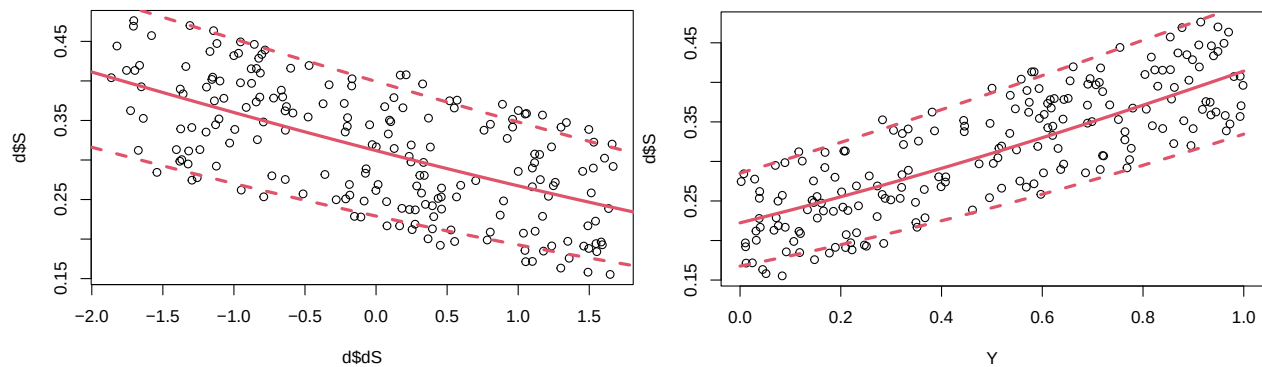
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided data

```
post$SL[1,] == post$S[1,]
```

```
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

```
plot(d$dS, d$S)
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function(i) inv_logit(post$aS + rep(post$bS_dS, 25) * i + post$bS_Y * colMeans(post$Y))) # Used me
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

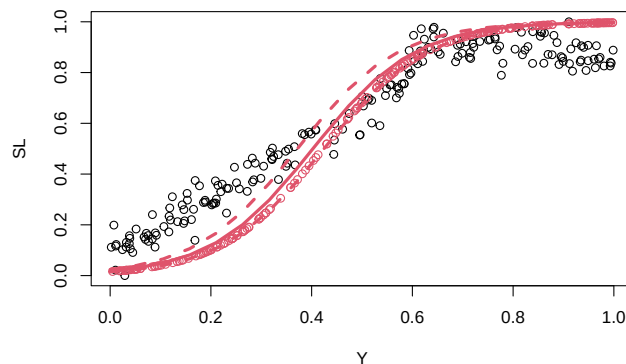
plot(Y, d$S)
Yseq<-seq(0, 1, len = 20)
SbyY<- sapply(Yseq,
  function(i) inv_logit(post$aS + rep(post$bS_dS, 25) * d$dS + post$bS_Y * i))
means<- apply( SbyY , 2 , mean )
PIs<- apply( SbyY , 2 , PI )
lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



- Functions catch the trend very well!

## Model SL by Y

```
plot(Y, SL)
points(post$Y[1,], post$SL[1,], col = 2)
Yseq<-seq(0, 1, len = 20)
SLbyY<- sapply(Yseq,
  function (i) inv_logit(post$aSL + post$bSL * i))
PIs<- apply( SLbyY , 2 , PI )
means<- apply( SLbyY , 2 , mean )
lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



- The model 1 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.
- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

## Model 2 (m2)

- see priors in overview.pdf, but this include correlation between SL and S
- warnings are annoying but relatively few.

```
# Model 2
m2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
```

```

logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

# We do not know anything about this but think similar for all group.
# So use partial pooling to regularize the estimate.
# mean should start at 0 and has small variation.

# In this case (m2), we do include correlation between S and SL due to Y.
vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
c(aK_bar, bK_Y_bar)~normal(0,0.3),
c(sigma_aK, sigma_bK_Y) ~ exponential(6),
z_aK[T]~normal(0,0.01),
z_bK_Y[T] ~ normal(0,0.01),

transpars>vector[25]:bK_S <- v[,1],
transpars>vector[25]:bK_SL <- v[,2],

matrix[25, 2]:v ~ multi_normal(c(bK_S_bar, bK_SL_bar), RhoSSL, sigmaSSL),
c(bK_S_bar, bK_SL_bar) ~ normal(0, 0.3),
corr_matrix[2]:RhoSSL ~ lkj_corr(1),# this is not too strong but good correlation
vector[2]: sigmaSSL ~ exponential(2),

# don't need regularization
# We are hoping/believing b_dK is negative, but this is not prior.
bK_dK ~ normal(0, 0.5),
U ~ exponential(1),

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_dS * dS,

# I am hoping/believing b_dS is negative and b_Y_S is positive
# partial pooling for the same reason for a2 and b_Y_S
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],

c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
c(sigma_aS, sigmabS_Y) ~ exponential(6),
z_aS[T] ~ normal(0,0.01),
zbS_Y[T] ~ normal(0,0.01),

# don't need regularization
bS_dS ~ normal(0,0.1),
sigma_S ~ exponential(1),

# Model SL by Y
# The log function and prior was refered to the previous research of Sea Level Change over time
# This function is abstract so I will have to actually stan code to provide accurate estimates
# ulam code in r does not allow us to hard code the conditional function.
# vector[200]: SL ~ normal(mu_SL + aSL2, sigmaSL),
vector[200]: SL ~ normal(mu_SL, sigma_SL),
vector[200]: logit(mu_SL) <- aSL + Y[i] * bSL,
# Should be very low variation since this part is hard coded.

```

```

# This time the estimation is not great so this is fine
aSL ~ normal(-4, 0.2),
bSL ~ normal(10, 0.2),

sigma_SL ~ exponential(1),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ uniform( 0 , 1 )

), dat = d, constraints = list(
  SL = "lower=0,upper=1" # SL is between 0 and 1
),chains=4, cores=4, log_lik=TRUE
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:1966:3: note: by 'void ulam_cm
## 1966 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:1966:3: note: by 'void ulam_cm
## 1966 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##      | ~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note: by 'T_ ulam_cmds
## 2005 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note: by 'T_ ulam_cmds
## 2005 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note: by 'T_ ulam_cmds
## const'
## 2005 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,

```

```

##          |          ^~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## 2005 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##          |   ^~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a0233e449c.hpp:2005:3: note:   by 'T_ ulam_cmds
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance parameters, this is usually not a problem. It may happen if the model is severely ill-conditioned or if the
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the

```

```

## Chain 1 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 2 Iteration: 1 / 1000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of th

```



```

## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration:    1 / 1000 [  0%]  (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the

```



```

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_lpdf: Correlation matrix is not positive definite. (in 'C:/Users/nobut/A
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4

## Chain 1 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%] (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%] (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%] (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%] (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 1 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 2 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 4 Iteration: 500 / 1000 [ 50%] (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%] (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%] (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 3 finished in 82.8 seconds.
## Chain 2 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%] (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%] (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%] (Sampling)

```

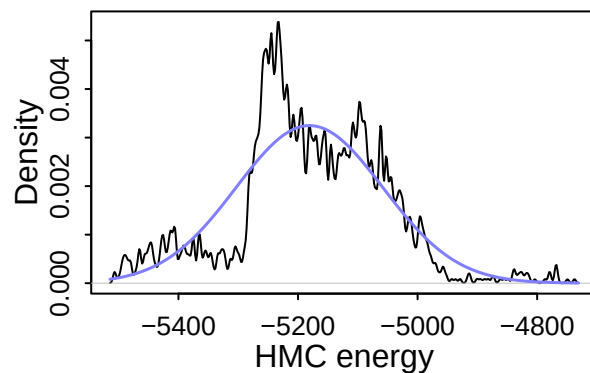
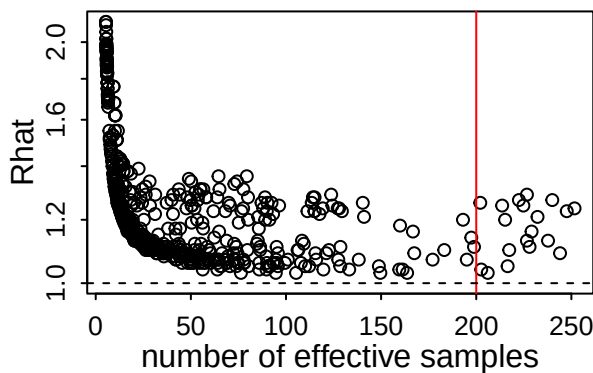
```
## Chain 1 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%] (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 2 finished in 118.1 seconds.
## Chain 1 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 1 finished in 119.9 seconds.
## Chain 4 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 4 finished in 120.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 110.2 seconds.
## Total execution time: 120.2 seconds.

## Warning: 354 of 2000 (18.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 1646 of 2000 (82.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.

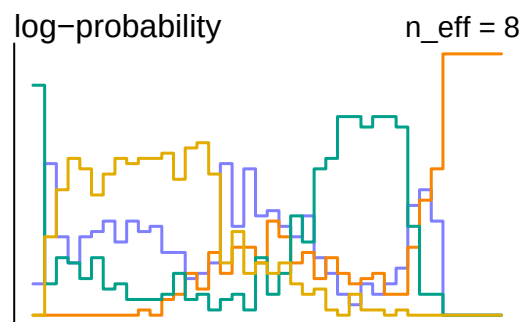
## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

`dashboard(m2)`



**354**  
Divergent transitions

Check yourself before  
you wreck yourself



`precis(m2, depth=1)`

```
## 606 vector or matrix parameters hidden. Use depth=2 to show them.
##
##          mean          sd      5.5%      94.5%      rhat
## bK_Y_bar  0.165780321 0.2019635728 -0.1481178677  0.493817843 2.097281
## aK_bar   -1.911070662 0.0409253628 -1.9789070905 -1.846797967 1.168399
## sigma_bK_Y 0.152991015 0.1530166528 0.0113662597 0.445306655 1.190466
## sigma_aK   0.170641750 0.1545570564 0.0151426695 0.477064797 1.214968
## bK_SL_bar  0.635994421 0.1439764971 0.3734759905 0.834801737 2.058359
```

```
## bK_S_bar      1.003825779 0.1715092918  0.7698341854  1.250929127 1.547099
## bK_dK        -0.208654745 0.0141612870 -0.2298456284 -0.187076440 1.241741
## U            0.026015496 0.0016418938  0.0232596758  0.028690889 1.253577
## bS_Y_bar     -0.008555264 0.2737191049 -0.4696354939  0.411995506 1.056501
## aS_bar       -0.791904787 0.0193676505 -0.8223920344 -0.760706391 1.173540
## sigmabS_Y    0.195896961 0.1660871273  0.0102828062  0.469428674 1.070791
## sigma_aS     0.161585985 0.1910519476  0.0050833270  0.513328477 1.267983
## bS_dS        -0.217823423 0.0181582676 -0.2474557245 -0.188783996 1.198626
## sigma_S      0.061184056 0.0028753852  0.0573485297  0.066182569 1.077268
## aSL          -3.936998983 0.1436936362 -4.1818225990 -3.720997135 1.278650
## bSL          10.009196070 0.1903250200  9.6819517270 10.237138785 1.139508
## sigma_SL     0.001379435 0.0008241549  0.0004333851  0.002727129 1.462031
##              ess_bulk
## bK_Y_bar      5.426178
## aK_bar       18.036671
## sigma_bK_Y   75.169834
## sigma_aK     141.018583
## bK_SL_bar     5.539528
## bK_S_bar     10.121315
## bK_dK        12.439204
## U            28.513402
## bS_Y_bar     99.452675
## aS_bar       44.327135
## sigmabS_Y    49.022474
## sigma_aS     53.670231
## bS_dS        193.034621
## sigma_S      53.236920
## aSL          11.822979
## bSL          19.702970
## sigma_SL     8.013526
```

-> Looks worse sampling efficiency.

## Posterior Check

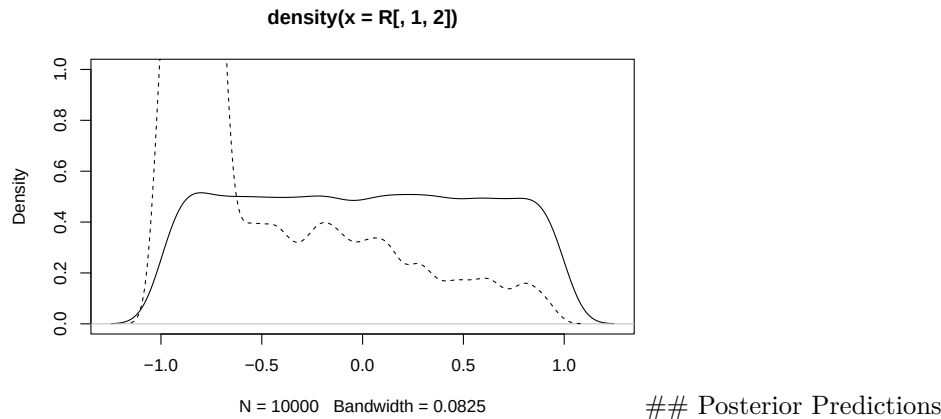
```
post <- extract.samples(m2)
```

- I do not check the Hidden parameters since I have never touched it since m1

## Correlation?

Rho value shows strong negative correlation between them.

```
R <- rljcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1))
dens(post$Rho[,1,2], add=TRUE, lty=2) # low correlation
```



```
plot(d$S, d$K)
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
  function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + t(t(post$bK_Y) * colMeans(post$SL)))
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K)
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + t(t(post$bK_Y) * colMeans(post$SL)))
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

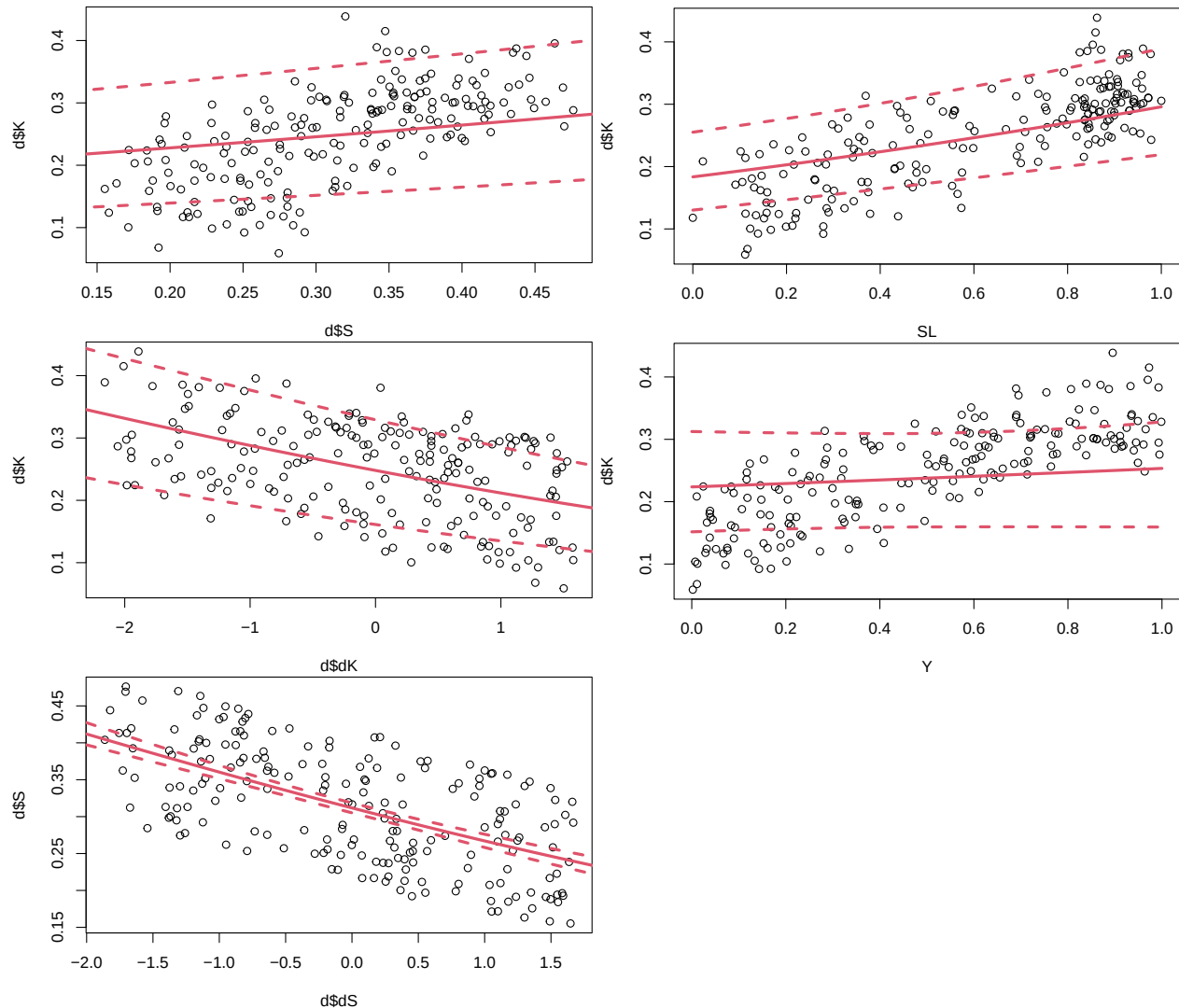
plot(d$dK, d$K)
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + t(t(post$bK_Y) * colMeans(post$SL)))
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(Y, d$K)
Yseq<-seq(0, 1, len = 20)
KbyY<- sapply(Yseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + t(t(post$bK_Y) * colMeans(post$SL)))
PIs<- apply( KbyY , 2 , PI )
means<- apply( KbyY , 2 , mean )
lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```

```

# Model S
plot(d$dS, d$dK)
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```



K estimation by SL is a bit narrower than it supposed to be but other than that, everything looks great. Let me compare those two models

```

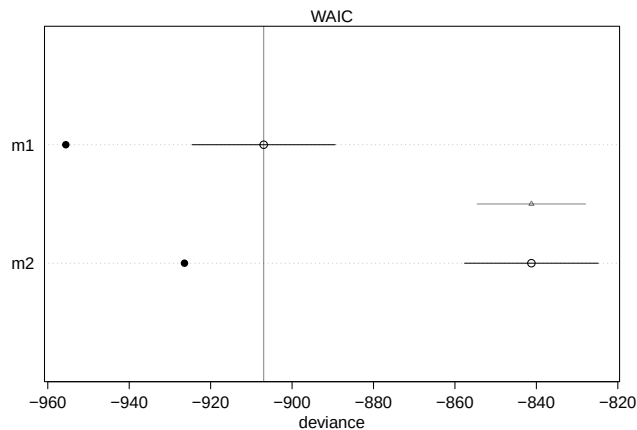
# Compare models
model_comparison <- compare(m1, m2)
print(model_comparison)

```

|    | WAIC      | SE       | dWAIC   | dSE | pWAIC    | weight       |
|----|-----------|----------|---------|-----|----------|--------------|
| m1 | -906.9690 | 17.57863 | 0.00000 | NA  | 24.30153 | 1.000000e+00 |

```
## m2 -841.2181 16.42068 65.75085 13.33656 42.60475 5.276956e-15
```

```
# Plot WAIC/PSIS differences
plot(model_comparison)
```



### Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

Therefore, the m1 is better than the other one.

## Model 3

```
m3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],

    # We do not know anything about this but think similar for all group.
    # So use partial pooling to regularize the estimate.
    # mean should start at 0 and has small variation.

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.3),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.3), # regularization
    sigma_bK_T ~ exponential(6),
    z_bK_T[T] ~ normal(0, 1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],
```

```

# I am hoping/believing b_dS is negative and b_Y_S is positive
# partial pooling for the same reason for a2 and b_Y_S
c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
bS_T_bar ~ normal(0, 0.001), # regularization
sigma_bS_T ~ exponential(10),
z_bS_T[T] ~ normal(0, 0.1),

sigma_S ~ exponential(1),

# Model SL by Y
# The log function and prior was referred to the previous research of Sea Level Change over time
# This function is abstract so I will have to actually stan code to provide accurate estimates
# ulam code in r does not allow us to hard code the conditional function.
# vector[200]: SL ~ normal(mu_SL + aSL2, sigma_SL),
vector[200]: SL ~ normal(mu_SL, sigma_SL),
vector[200]: logit(mu_SL) <- aSL + Y[i] * bSL,
# Should be very low variation since this part is hard coded.
# This time the estimation is not great so this is fine
aSL ~ normal(-4, 0.2),
bSL ~ normal(10, 0.2),

sigma_SL ~ exponential(1),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ uniform( 0 , 1 )

), dat = d, constraints = list(
  SL = "lower=0,upper=1" # SL is between 0 and 1
),chains=4, cores=4, log_lik=TRUE
)

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ^~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1458:3: note: by 'void ulam_cm
## 1458 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ^~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ^~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1458:3: note: by 'void ulam_cm
## 1458 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ^~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>
## & theta,
## | ^~~~~~

```

```

## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1495:3: note:   by 'T_ ulam_cmds
## 1495 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 |   inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1495:3: note:   by 'T_ ulam_cmds
## 1495 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1495:3: note:   by 'T_ ulam_cmds
## l_namespace::ulam_cmdstanr_e8808b3bec7cf7eae33015ed0d861c44_model::log_prob(Eigen::Matrix<T_a, -1, 1>
## 1495 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 |   inline double log_prob(Eigen::VectorXd& theta,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpIVVMbi/model-28a07e285ac1.hpp:1495:3: note:   by 'T_ ulam_cmds
## 1495 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 1
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 2
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 3
## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)

```





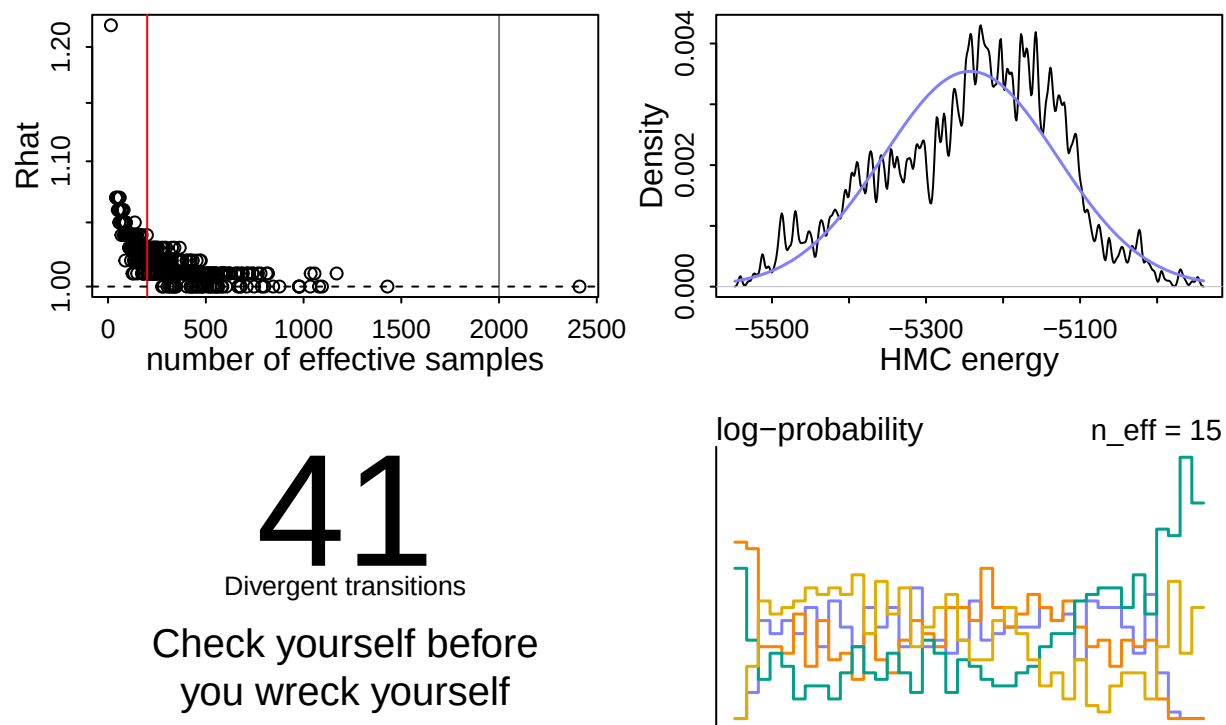
```
## Chain 4 Iteration: 1000 / 1000 [100%] (Sampling)
## Chain 4 finished in 78.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 76.9 seconds.
## Total execution time: 78.5 seconds.

## Warning: 41 of 2000 (2.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 1959 of 2000 (98.0%) transitions hit the maximum treedepth limit of 10.
## See https://mc-stan.org/misc/warnings for details.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m3)
```



```
precis(m3, depth=1)
```

```
## 450 vector or matrix parameters hidden. Use depth=2 to show them.

##           mean          sd        5.5%        94.5%        rhat
## bK_dK      -0.2222348571  0.0100881546 -0.2378321195 -0.206314163  1.007674
## bK_Y        0.4814524393  0.1158737726  0.3001090350  0.664924171  1.021572
## bK_SL       0.4390938228  0.0988893395  0.2790571018  0.595327412  1.029505
## bK_S        0.5727260377  0.1869619437  0.2659714224  0.872652075  1.007197
## aK         -0.9192406714  0.2145098752 -1.2696513835 -0.570300573  1.010438
## bK_T_bar   -0.8935200101  0.2163480659 -1.2408578780 -0.548674731  1.015770
## sigma_bK_T  0.0381912713  0.0193872098  0.0082236820  0.070213589  1.017090
## U          0.0236405940  0.0013736800  0.0215053150  0.025927121  1.006143
## bS_dS      -0.2196797713  0.0069778985 -0.2308786657 -0.208373387  1.018823
## bS_Y        0.9169159711  0.0288369313  0.8684379450  0.960211646  1.069865
## aS         -1.2593804232  0.0199121692 -1.2879606355 -1.224725129  1.061922
```

```
## bS_T_bar    -0.0001339781 0.0009733411 -0.0017063058 0.001438286 1.000173
## sigma_bS_T  0.1350977089 0.0946675067 0.0142122521 0.305358071 1.039579
## sigma_S     0.0107480053 0.0017658000 0.0080783996 0.013728326 1.058554
## aSL         -4.0162581595 0.1688307925 -4.2862777015 -3.747856299 1.065809
## bSL         9.9142919043 0.2007850630 9.5716602370 10.227579150 1.050495
## sigma_SL    0.0021137159 0.0011652257 0.0007608262 0.004240901 1.220162
##            ess_bulk
## bK_dK       532.60845
## bK_Y        469.89497
## bK_SL       286.37287
## bK_S        465.54908
## aK          480.74274
## bK_T_bar    421.33381
## sigma_bK_T  170.76910
## U           590.67749
## bS_dS       152.69735
## bS_Y        51.44444
## aS          79.64208
## bS_T_bar    2411.67265
## sigma_bS_T  119.43479
## sigma_S     70.60043
## aSL         59.62564
## bSL         57.18123
## sigma_SL    14.59815
```

-> Looks worse sampling efficiency.

## Posterior Check

```
post <- extract.samples(m3)
```

- Again, I do not check the Hidden parameters since I have never touched it since m1

```
# Model K
```

```
plot(d$S, d$K)
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
  function(i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_Y * colMe
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K)
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq, # This is tricky but use bK_T for T value and others are not stratified so random
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_Y * colMeans(post$Y) +
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
```

```

lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$dK, d$K)
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_Y * col
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(Y, d$K)
Yseq<-seq(0, 1, len = 20)
KbyY<- sapply(Yseq,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_Y * i +
PIs<- apply( KbyY , 2 , PI )
means<- apply( KbyY , 2 , mean )
lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$K)
KbyT<- sapply(1:25,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_Y * col
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i])), col = 2)

# Model S

plot(d$dS, d$S)
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + post$bS_dS * i + post$bS_Y * colMeans(post$Y) + colMeans(post$bS_T
means<- apply( SbydS , 2 , mean )
PIs<- apply( SbydS , 2 , PI )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(Y, d$S)
Yseq<-seq(0, 1, len = 20)
SbyY<- sapply(Yseq,
  function (i) inv_logit(post$aS + post$bS_dS * d$dS + post$bS_Y * i + colMeans(post$bS_T)[d$T]))
means<- apply( SbyY , 2 , mean )
PIs<- apply( SbyY , 2 , PI )

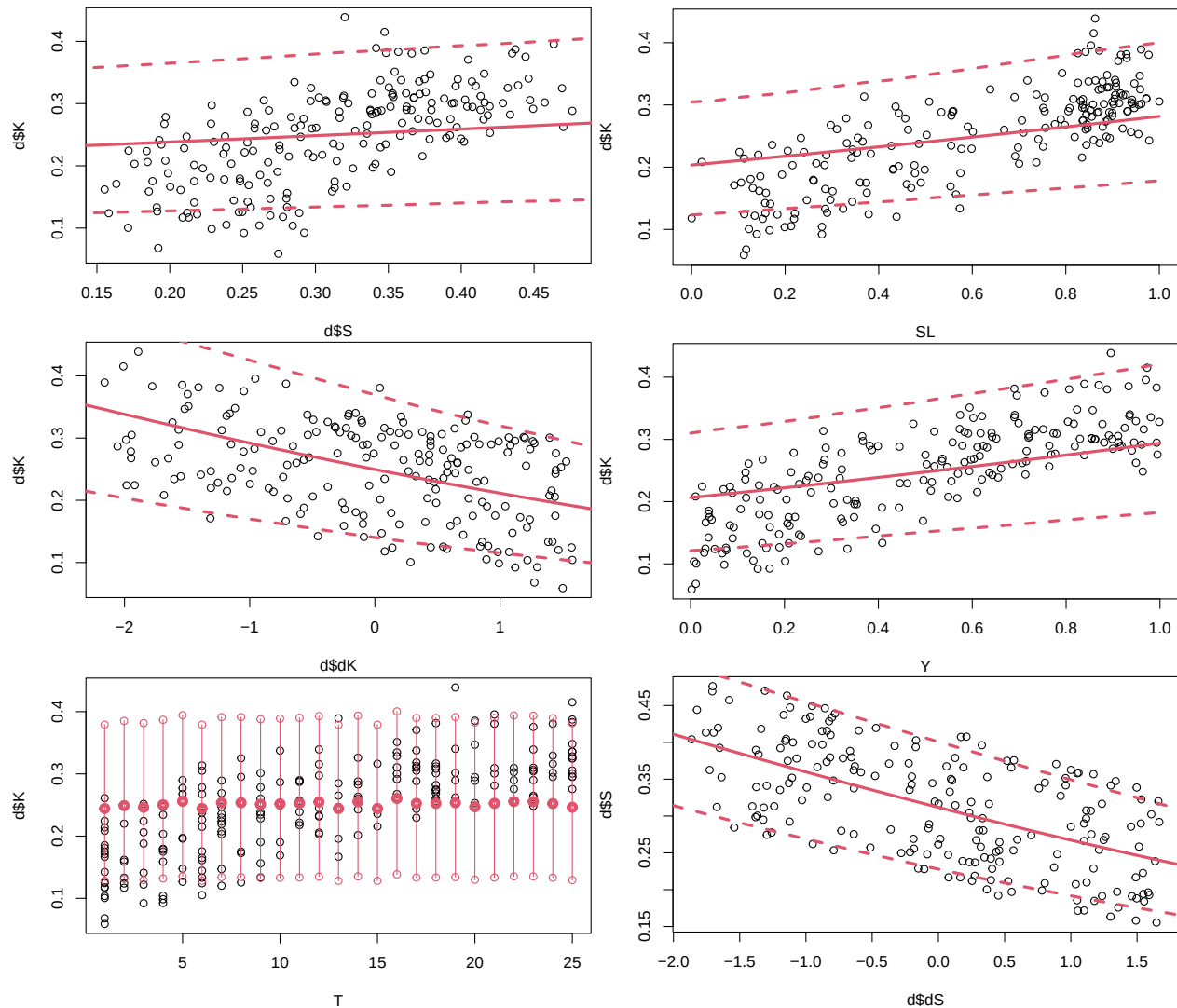
```

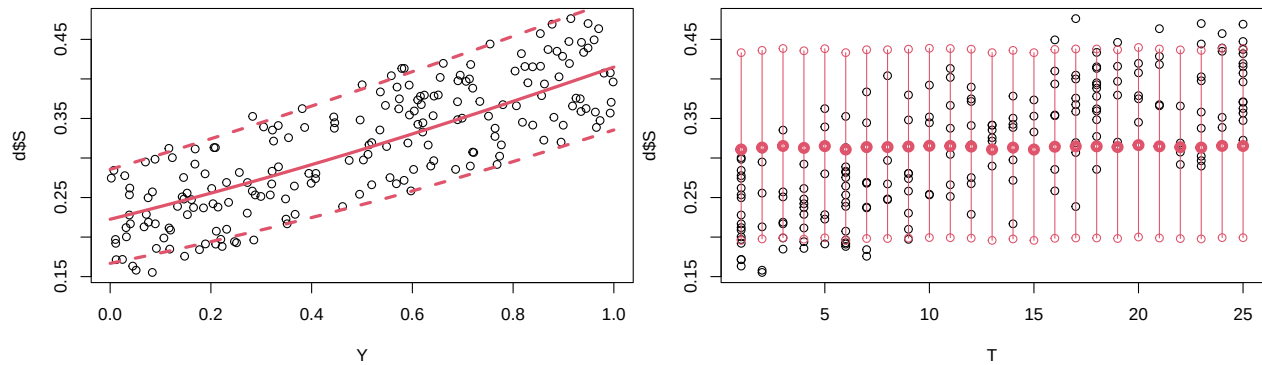
```

lines(Yseq, means, col = 2, lwd = 3)
lines(Yseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Yseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$S)
SbyT<- supply(1:25,
  function(i) inv_logit(post$aS + post$bS_dS * d$dS + post$bS_Y * colMeans(post$Y) + colMeans(post$b
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i])), col = 2)

```





I have no idea why it does not model bK\_T and bS\_T vvery well... I will figure it out. Other than that, looks great, let me compare all those models.

```
# Compare models
```

```
model_comparison <- compare(m1, m3)
```

```
print(model_comparison)
```

```
##           WAIC      SE   dWAIC      dSE   pWAIC   weight
## m1 -906.9690 17.57863 0.000000      NA 24.30153 0.8925166
## m3 -902.7356 17.56686 4.233418 2.246634 25.72843 0.1074834
```

```
# Plot WAIC/PSIS differences
```

```
plot(model_comparison)
```

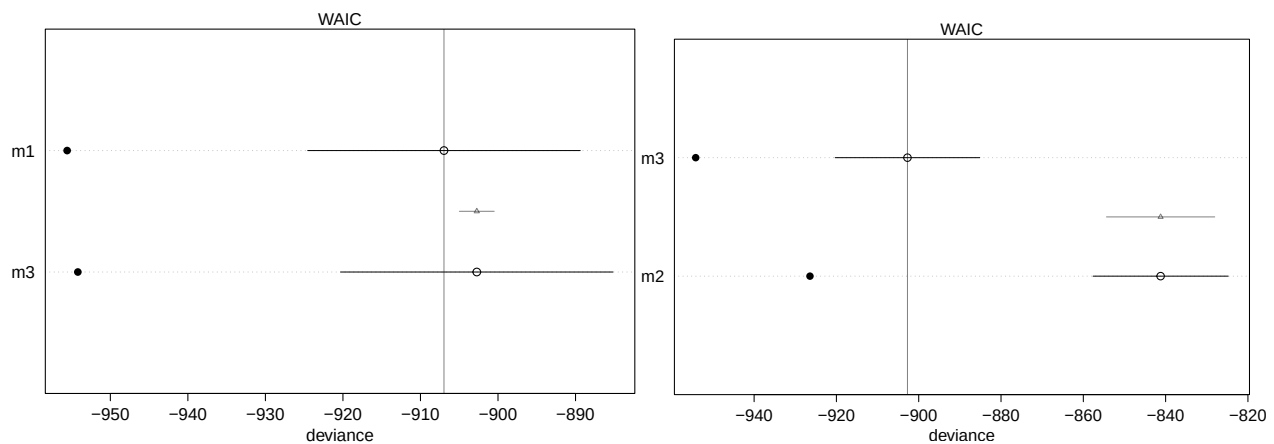
```
model_comparison <- compare(m2, m3)
```

```
print(model_comparison)
```

```
##           WAIC      SE   dWAIC      dSE   pWAIC   weight
## m3 -902.7356 17.56686 0.000000      NA 25.72843 1.00000e+00
## m2 -841.2181 16.42068 61.51744 13.15864 42.60475 4.38186e-14
```

```
# Plot WAIC/PSIS differences
```

```
plot(model_comparison)
```



This looks Model 1 and 3 are almost same, but model 3 is a bit better.

## Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K) may be examined with actual data.

- The Model 3 still has a problem of bK\_T and bS\_T as they do not fit the data. I am trying to understand the reason and fix it.

```
rm(list=ls())
```